Assignment 2 — Solutions

Part I — Short Answer / Reasoning

Q1.

The query returns 0 because `NULL` represents an unknown value, and any comparison using `=` against `NULL` evaluates to `UNKNOWN` — never `TRUE`. A `WHERE` clause keeps only rows that evaluate to `TRUE`, so no rows qualify. Corrected query:

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

Q2.

Yes, the student is correct. After GROUP BY department, the result already contains exactly one row per department, so every `(department, COUNT(*))` pair is guaranteed unique. DISTINCT removes duplicate rows, but they will not be present once the rows are grouped

Q3.

LEFT JOIN will always return more rows than the corresponding INNER JOIN. In this case LEFT JOIN having more rows shows that there are customers present who don't have any orders yet, and therefore will be assigned to NULL values in the LEFT JOIN

Q4.

The query is invalid because `employee_name` appears in `SELECT` but is neither listed in `GROUP BY` nor wrapped in an aggregate. Grouping by `department` collapses many employees into a single row per department, so there is no one `employee_name` for the group. If SQL "allowed" it, the engine would have to pick a name arbitrarily from the many in each group, producing an undefined / non-deterministic value. Valid correction:

One row per department:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

Q5.

The error is using the aggregate `AVG(salary)` inside `WHERE`. The logical execution order is `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`. `WHERE` runs before grouping, so per-group aggregates do not exist yet. Filtering on an aggregate must use `HAVING`, which runs after groups are formed:

```
SELECT department, AVG(salary)

FROM EmployeeGROUP

BY department

HAVING AVG(salary) > 80000;
```

Q6.

(a) Products priced above the overall average — non-correlated.

The overall average is a single scalar computed once across all products, independent of any outer row.

```
SELECT * FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

(b) Employees earning more than their own department's average — correlated.
The threshold depends on each employee's department, so the subquery must reference the outer row.

```
SELECT * FROM Employee e
WHERE salary > (SELECT AVG(salary) FROM Employee e2
                WHERE e2.department = e.department);
```

Part II — Long Answer (Food Delivery Schema)

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

Why: An `INNER JOIN` only returns customers who *do* have orders — the opposite of "never."

(b) Average order value per restaurant (> 5 orders)

```
SELECT restaurant_id, AVG(order_value) AS avg_order_value
FROM (
    SELECT o.order_id, o.restaurant_id,
           SUM(mi.price * oi.quantity) AS order_value
    FROM Orders o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id, o.restaurant_id
) AS order_values
GROUP BY restaurant_id
HAVING COUNT(*) > 5;
```

Why: The problem needs two aggregation levels — sum the items *within* each order, then average *across* orders — so a derived table computes each order's value first. `HAVING` (not `WHERE`) applies the `> 5` filter because the order count is an aggregate known only after grouping.

(c) Restaurants above the platform-wide average order value

```
WITH order_values AS (
    SELECT o.order_id, o.restaurant_id,
           SUM(mi.price * oi.quantity) AS order_value
    FROM Orders o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id, o.restaurant_id
),
```

```
restaurant_avg AS (
    SELECT restaurant_id, AVG(order_value) AS avg_order_value
    FROM order_values
    GROUP BY restaurant_id
    HAVING COUNT(*) > 5
)
SELECT restaurant_id, avg_order_value
FROM restaurant_avg
WHERE avg_order_value > (SELECT AVG(order_value) FROM order_values);
```

Why non-correlated: The platform-wide average is a single scalar computed once over all orders; it does not depend on which restaurant the outer query is currently evaluating, so no correlation is needed. A correlated subquery would only be required if the comparison value changed per outer row.

Interpretation note: "Platform-wide average order value" is read here as the average over all individual orders. If your course means the average of the per-restaurant averages, change the final subquery to `SELECT AVG(avg_order_value) FROM restaurant_avg`.

(d) Most ordered menu item per restaurant

```
WITH item_totals AS (
   SELECT mi.restaurant_id, oi.item_id, mi.item_name,
           SUM(oi.quantity) AS total_qty
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY mi.restaurant_id, oi.item_id, mi.item_name
),
ranked AS (
    SELECT restaurant_id, item_id, item_name, total_qty,
           RANK() OVER (PARTITION BY restaurant_id
                        ORDER BY total_qty DESC) AS rnk
    FROM item_totals
)
SELECT restaurant_id, item_id, item_name, total_qty
FROM ranked
WHERE rnk = 1;
```

A basic `GROUP BY + MAX` approach gives the maximum quantity per restaurant but not *which* item achieved it; recovering the item needs an extra self-join on the max value, which silently returns multiple rows on ties and does not generalize to top-N for N > 1.

Part III — Normalization

Q1. Functional dependencies

- `student_id` → student_name
- `course_id` → course_name, instructor_id
- `instructor_id` → instructor_name, instructor_office
- `(student_id, course_id)` → grade, enrollment_date

Candidate key: `(student_id, course_id)`. Transitively, `course_id → instructor_id → instructor_name, instructor_office`.

Q2. Is the table in 1NF?

Yes. Every column holds a single atomic value (no repeating groups, lists, or multi-valued cells) and the table has a defined primary key `(student_id, course_id)`. That satisfies 1NF.

Q3. Partial dependency (2NF violation)

2NF forbids a non-prime attribute depending on only part of the composite key. Here `student_id → student_name` and `course_id → course_name` (plus the instructor columns via `course_id`) each depend on only half of the key `(student_id, course_id)`.

Update anomaly: Consider the course name “Databases” is stored once per enrolled student; renaming it to “Database Systems” requires updating every such row, and missing one leaves the data inconsistent.

Insertion anomaly: a new course cannot be recorded until at least one student enrolls, because the primary key needs a `student_id`.

Q4. Transitive dependency (3NF violation)

`course_id → instructor_id → instructor_name, instructor_office`. The non-prime attributes `instructor_name` and `instructor_office` depend on `instructor_id`, which is itself a non-prime attribute rather than the key.

Anomalies: if an instructor changes office, every enrollment row mentioning that instructor must be updated; and if the last course taught by an instructor is deleted, that instructor's office information disappears entirely (deletion anomaly).

Q5. 3NF decomposition

Primary keys are shown in bold; foreign keys are noted in the justification.

`Student(`**`student_id`**`, student_name)`
Justified by: student_id → student_name.

`Instructor(`**`instructor_id`**`, instructor_name, instructor_office)`
Justified by: instructor_id → instructor_name, instructor_office.

`Course(`**`course_id`**`, course_name, instructor_id (FK))`
Justified by: course_id → course_name, instructor_id; instructor_id is a foreign key to Instructor.

`Enrollment(`**`student_id`**`, `**`course_id`**`, grade, enrollment_date)`

Justified by: (student_id, course_id) → grade, enrollment_date; both key columns are foreign keys to Student and Course.

No non-prime attribute now depends on part of a key or transitively through another non-prime attribute, so all four relations are in 3NF (and in fact BCNF).

Part IV — Design Justification

In Part II(a), I could have expressed customers who never placed an order using either a `LEFT JOIN` with a `NULL` check or a `NOT EXISTS` subquery. I chose the `LEFT JOIN` form because it makes the missing-match idea visible: preserve every customer, then keep only rows where no order joined. It is also readable for this schema because `order_id` is a primary key, so checking `o.order_id IS NULL` cannot accidentally match a real order. A `NOT EXISTS` version would be equally valid and can be preferable when duplicate rows or nullable join columns make an anti-join harder to reason about; many optimizers convert both forms to similar plans.

Normalizing the Enrollment table removes redundancy and the update / insert / delete anomalies, but it trades that against query convenience. A question like “show each student's name, course name, instructor office, and grade” now requires joining four tables instead of reading one, which costs both performance and query complexity. Real systems sometimes keep controlled denormalization — especially read-heavy reporting tables and data warehouses — to avoid expensive multi-table joins, accepting redundant copies and the burden of keeping them consistent in exchange for faster reads. OLTP systems favour normalization for integrity, while OLAP / analytics workloads often denormalize for speed.